PROJECT-BASED LEARNING (PBL) REPORT

DATABASE MANAGEMENT SYSTEMS

Small Database Application Development using Python



PROJECT TITLE

LEGAL CASE AND CLIENT MANAGEMENT SYSTEM

JurisDesk

Submitted by

Praveen Choudhary

Roll Number: 25WU0102215

Faculty: C Nagaraju

Programme: B.Tech CSE (Artificial Intelligence and Machine Learning)

Section: AIML Leopards

School of Technology | Woxsen University

Academic Year: 2026–27

Date of Submission: 12/10/2026

UNDERTAKING

I hereby declare that the project entitled “Legal Case and Client Management System (JurisDesk)”, submitted as part of the Project-Based Learning activity for the Database Management Systems course, has been prepared for academic demonstration under the guidance of C Nagaraju. The report describes the relational design and local Python Flask and SQLite prototype available with the project files. Sample client and case records are fictional demonstration data. I acknowledge the use of relevant technical documentation and take responsibility for reviewing the report against the submitted source code before final submission.

Name: Praveen Choudhary

Roll Number: 25WU0102215

Programme: B.Tech CSE (AI & ML)

Date: 12/10/2026 Place: Hyderabad

Signature: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing the opportunity to undertake this Project-Based Learning activity as part of the Database Management Systems course. I thank my course faculty, C Nagaraju, for guidance and feedback that helped connect database concepts with an application-oriented problem.

I also acknowledge the Python, Flask and SQLite documentation used to understand routing, database connectivity, relational constraints and query execution. The project helped me practise data modelling, primary and foreign keys, normalization, SQL joins, aggregate queries and application-level validation.

The sample legal records included in the prototype are illustrative and do not represent actual clients, lawyers or legal proceedings.

TABLE OF CONTENTS

Contents	Page
Undertaking	2
Acknowledgement	3
Abstract and Keywords	7
List of Tables	5
List of Figures	6
List of Acronyms	6
1. Problem Statement, Solution Strategy and ER Diagram	8
1.1 Problem Statement and Objectives	8
1.2 Scope and Assumptions	9
1.3 Proposed Solution Strategy	9
1.4 Functional Requirements	10
1.5 ER Diagram and Relationships	11
2. Logical Database Design	12
2.1 Relational Schema	12
2.2 Keys and Integrity Constraints	13
2.3 Functional Dependencies	14
2.4 Normalization up to 3NF	14
2.5 Data Dictionary	15
2.6 DDL and Sample Data	17
3. Application Development	19
3.1 Application Overview	19
3.2 Development Environment and Connectivity	20
3.3 Module-to-Table Mapping	20
3.4 CRUD Operations	21
3.5 Business Rules and Validation	21
3.6 Reports and SQL Operations	22
3.7 Application Screens	23
3.8 Testing and Results	24
4. Conclusion and Future Enhancements	26
5. References	27
6. Appendices	28

LIST OF TABLES

Table No.	Title	Page
1	Scope and Assumptions	9
2	Proposed Solution Strategy	9
3	Functional Requirements	10
4	Entity and Relationship Summary	11
5	Relational Schema	12
6	Keys and Integrity Constraints	13
7	Normalization Summary	14
8	Data Dictionary	15
9	Database Environment	17
10	Application Environment	19
11	Module-to-Table Mapping	20
12	CRUD Operations	21
13	Business Rules and Validation	21
14	Reports and SQL Operations	22
15	Test Cases	24

LIST OF FIGURES AND ACRONYMS

Figure No.	Figure Title	Page
1	JurisDesk ER Diagram	11
2	Application Architecture	20
3	Dashboard and module screens	23
4	Database inspection example	23

List of Acronyms

Acronym	Expansion
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
ERD	Entity-Relationship Diagram
FK	Foreign Key
HTML	HyperText Markup Language
PK	Primary Key
SQL	Structured Query Language
UI	User Interface
UK	Unique Key

ABSTRACT

JurisDesk is a small database-backed application designed to organize legal case and client information for an academic demonstration. Legal offices need to maintain client contacts, case details, lawyer assignments, hearing dates, tasks, invoices and payments. Disconnected spreadsheets can lead to repeated data, missed deadlines and difficulty determining outstanding balances. The project uses Python, Flask and SQLite to provide a local web interface over a relational database. The schema contains entities for clients, lawyers, case types, courts, cases, assignments, hearings, tasks, documents, time entries, expenses, invoices and payments. Primary keys identify records, foreign keys connect related entities, and selected UNIQUE and CHECK constraints protect data quality. The application includes forms and lists for clients, lawyers, cases, assignments, hearings, tasks, billing and payments, together with summary reports. Application logic prevents overlapping hearings for the same lawyer, duplicate lawyer-case assignments, duplicate invoice numbers and payments that exceed an invoice's outstanding balance. The report documents the problem, scope, entity relationships, logical schema, functional dependencies, normalization, SQL operations, application architecture and testing plan. The prototype uses fictional demonstration data and is intended for local academic use, not as a secure production system for confidential legal information.

Keywords: Legal case management, client records, SQLite, Flask, relational database, billing.

1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

A legal office maintains information about clients, cases, lawyers, court locations, hearings, work tasks and financial transactions. If these records are stored in separate spreadsheets or documents, the same details may be re-entered in multiple places. Staff may have difficulty finding all information related to a case, tracking upcoming hearings, identifying tasks that are still pending or calculating the balance on an invoice.

The problem addressed by JurisDesk is the organization of these related records in a single relational database and the provision of a simple interface for common case-management workflows. The prototype centralizes records and applies constraints and validation to reduce avoidable inconsistencies.

1.2 Objectives

- Design a relational database for clients, lawyers, cases, hearings, tasks and billing.
- Connect cases to clients, case types and courts through foreign keys.
- Provide local screens for registering and viewing key records.
- Detect overlapping hearing times for the same lawyer.
- Track invoice amounts, payments and outstanding balances.
- Demonstrate SQL joins, aggregates, constraints and application/database connectivity.

1.3 SCOPE AND ASSUMPTIONS

The scope covers a local demonstration application with fictional data. It supports core record creation and listing workflows and several summary reports. It is not intended to provide legal advice, electronic court filing, digital signatures or production-grade document storage.

Item	Description
In scope	Client and lawyer registers; case registration; case assignments; hearing scheduling; task tracking; invoices; payments; summary reports.
Schema-supported entities	Documents, time entries and expenses are present in the database schema; corresponding full UI workflows are not confirmed in the current interface.
Out of scope	Court integration, e-filing, secure document upload, authentication, multi-user hosting, automated legal research and notifications.
Assumptions	The app runs locally; sample records are illustrative; users enter valid dates and monetary values; the SQLite file remains with app.py.
Constraints	A single local SQLite database is used; no formal backup workflow or audit log is implemented.

1.4 Stakeholders

The primary user is a legal-office staff member or demonstrator who registers clients, maintains case details, schedules hearings and monitors tasks and invoices. Lawyers are represented as records and are associated with cases and hearings. Clients are represented as records linked to cases. The prototype does not implement separate login roles for these stakeholders.

1.5 PROPOSED SOLUTION STRATEGY

Solution element	Project-specific description
Interface	Flask-rendered HTML templates styled with a shared CSS file.
Application logic	Python route handlers receive form input, validate selected rules and execute SQL.
Database	SQLite file named legal_management.db located alongside app.py.
Relational design	Separate tables reduce repeated client, lawyer, court and case-type details.
Validation	Foreign keys, UNIQUE constraints, CHECK constraints and route-level checks.
Outputs	Dashboard counts, upcoming hearings, case-status counts, lawyer workload and outstanding invoices.

1.6 Functional Requirements

ID	Requirement	Priority
FR-01	Register and list clients.	High
FR-02	Register and list lawyers with specialization and hourly rate.	High
FR-03	Register cases with unique case number and linked client, type and optional court.	High
FR-04	Assign a lawyer to a case and prevent duplicate case-lawyer pairs.	High
FR-05	Schedule hearings and reject overlapping hearings for the same lawyer.	High
FR-06	Create tasks with due date, status and priority.	High
FR-07	Create invoices and show paid amount and outstanding balance.	High
FR-08	Record payments only when positive and not above the remaining balance.	High
FR-09	Display summary reports for case status, hearings and outstanding invoices.	Medium

1.7 ENTITY-RELATIONSHIP DIAGRAM

Figure 1 presents the principal entities in the current schema. Cases act as a central entity connected to clients, case types and courts. Assignments associate lawyers with cases. Hearings and tasks belong to cases; invoices belong to cases and payments belong to invoices. The schema also defines documents, time entries and expenses for future or extended workflows.

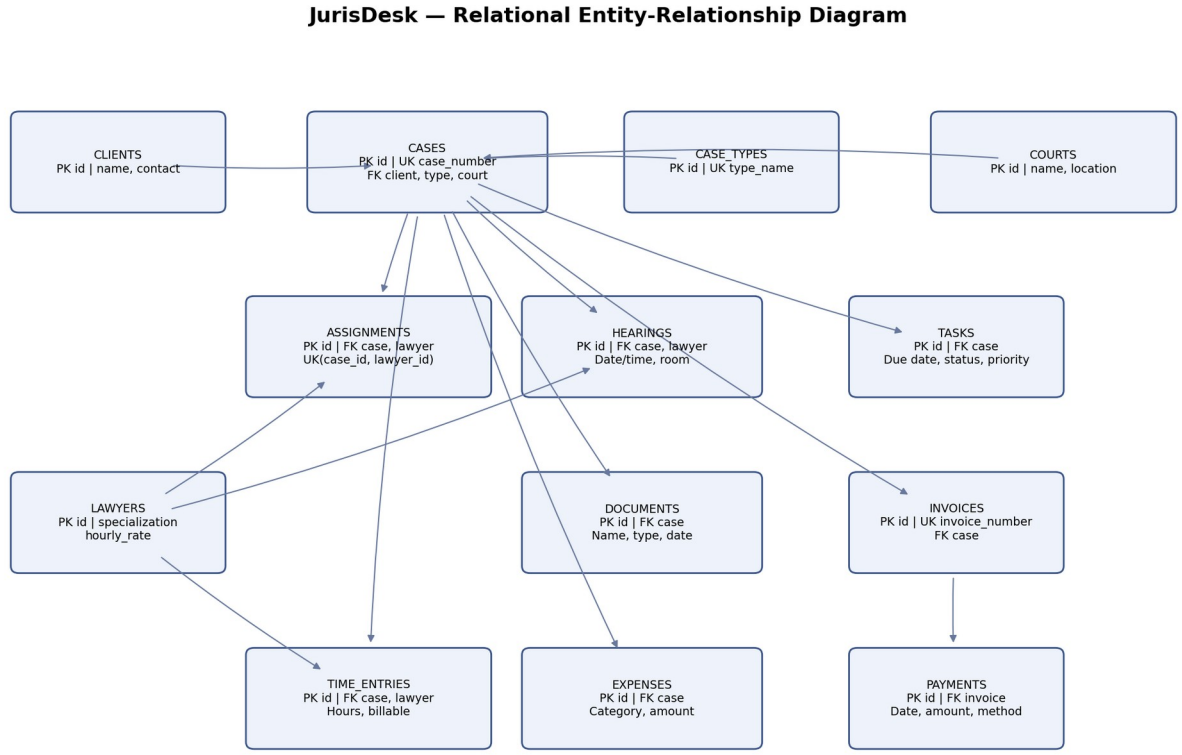


Figure 1. JurisDesk relational ER diagram, derived from the CREATE TABLE statements in app.py.

Entity / relationship	Purpose / cardinality
Client → Cases	One client may have many cases; each case references one client.
Case type → Cases	One type may classify many cases; each case references one type.
Court → Cases	One court may be linked to many cases; a case may have no court assigned.
Cases ↔ Lawyers	Many-to-many relationship implemented through assignments.
Case → Hearings / Tasks	One case can have multiple hearings and tasks.
Case → Invoices → Payments	A case may have multiple invoices; each invoice may have multiple payments.

2. LOGICAL DATABASE DESIGN

2.1 Relational Schema

The application creates thirteen tables through init_db(): clients, lawyers, case_types, courts, cases, assignments, hearings, tasks, documents, time_entries, expenses, invoices and payments. The schema separates descriptive master data from transactional records.

Relation	Primary key	Important foreign keys / attributes
clients	id	name, phone, email, address
lawyers	id	name, specialization, email, hourly_rate
case_types	id	type_name UNIQUE
courts	id	court_name, location
cases	id	case_number UNIQUE; client_id, case_type_id, court_id
assignments	id	case_id, lawyer_id; UNIQUE(case_id, lawyer_id)
hearings	id	case_id, lawyer_id; hearing_date, start_time, end_time
tasks	id	case_id; title, due_date, status, priority
documents	id	case_id; document_name, document_type, uploaded_on
time_entries	id	case_id, lawyer_id; hours, billable
expenses	id	case_id; category, amount
invoices	id	invoice_number UNIQUE; case_id, amount, status
payments	id	invoice_id; paid_on, amount, method

2.2 KEYS AND INTEGRITY CONSTRAINTS

Table	Key / constraint	Rule enforced
clients	PRIMARY KEY	Each client has a unique id.
lawyers	PRIMARY KEY; CHECK	Each lawyer has an id; hourly_rate must be non-negative.
case_types	UNIQUE(type_name)	Prevents duplicate case type labels.
cases	UNIQUE(case_number); FKs	Case number is unique; client and case type must exist.
assignments	UNIQUE(case_id, lawyer_id)	A lawyer cannot be assigned to the same case twice.
hearings	CHECK(end_time > start_time); FKs	End time must be later than start time; case and lawyer must exist.
tasks	FK case_id	Task must reference an existing case.
time_entries	CHECK(hours > 0); CHECK(billable IN (0,1))	Hours must be positive and billable is Boolean-like.
expenses	CHECK(amount >= 0)	Expense cannot be negative.
invoices	UNIQUE(invoice_number); CHECK(amount >= 0)	Invoice identifier is unique and amount non-negative.
payments	CHECK(amount > 0); FK invoice_id	Payment is positive and references an existing invoice.

Referential integrity

The connection helper executes PRAGMA foreign_keys=ON, which enables SQLite foreign-key enforcement for each connection. ON DELETE CASCADE is declared for case-dependent assignments, hearings and tasks, and for payments associated with an invoice. These rules help prevent orphaned dependent records when the supported deletion paths are used.

2.3 FUNCTIONAL DEPENDENCIES

Functional dependencies express how one attribute or a set of attributes determines other attributes. In the current design, the surrogate primary key of each entity determines its non-key attributes. Additional candidate keys exist where UNIQUE constraints are declared.

- clients.id → name, phone, email, address
- lawyers.id → name, specialization, email, hourly_rate
- case_types.id → type_name; case_types.type_name → id
- cases.id → case_number, title, client_id, case_type_id, court_id, status, opened_on, description
- assignments.id → case_id, lawyer_id, assigned_on; (case_id, lawyer_id) → assigned_on
- hearings.id → case_id, lawyer_id, court_room, hearing_date, start_time, end_time, notes
- invoices.id → invoice_number, case_id, issued_on, due_date, amount, status
- payments.id → invoice_id, paid_on, amount, method

2.4 Normalization up to Third Normal Form

First Normal Form (1NF): each field stores a single value and records are represented as rows. The schema does not intentionally store repeating lists inside a column.

Second Normal Form (2NF): tables use single-column surrogate primary keys, so non-key attributes depend on the whole primary key. The assignments table also has a composite UNIQUE constraint for case_id and lawyer_id to prevent a repeated relationship.

Third Normal Form (3NF): client details are stored in clients rather than repeated in cases; lawyer details are stored in lawyers rather than repeated in assignments and hearings; invoice information is separated from payment transactions. These separations reduce update, insertion and deletion anomalies.

Normalization assessment is based on the declared schema and functional dependencies. A full formal proof of BCNF is not claimed for every possible business dependency.

2.5 DATA DICTIONARY — MASTER TABLES

Table.attribute	Type / constraint	Description
clients.id	INTEGER PK	Client identifier.
clients.name	TEXT NOT NULL	Client or organization name.
clients.phone	TEXT	Optional contact number.
clients.email	TEXT	Optional email address.
clients.address	TEXT	Optional address.
lawyers.id	INTEGER PK	Lawyer identifier.
lawyers.name	TEXT NOT NULL	Lawyer name.
lawyers.specialization	TEXT NOT NULL	Legal specialization.
lawyers.email	TEXT	Optional email.
lawyers.hourly_rate	REAL NOT NULL DEFAULT 1000 CHECK >= 0	Hourly rate.
case_types.id	INTEGER PK	Case-type identifier.
case_types.type_name	TEXT UNIQUE NOT NULL	Classification label.
courts.id	INTEGER PK	Court identifier.
courts.court_name	TEXT NOT NULL	Court name.
courts.location	TEXT NOT NULL	Court location.

2.5 DATA DICTIONARY — CASE AND TRANSACTION TABLES

Table.attribute	Type / constraint	Description
cases.case_number	TEXT UNIQUE NOT NULL	Human-readable unique case reference.
cases.client_id	INTEGER FK NOT NULL	Client linked to case.
cases.case_type_id	INTEGER FK NOT NULL	Case classification.
cases.court_id	INTEGER FK nullable	Optional court reference.
cases.status	TEXT DEFAULT 'Open'	Case progress state.
assignments.case_id/lawyer_id	INTEGER FK; UNIQUE pair	Lawyer-case link.
hearings.hearing_date	TEXT NOT NULL	Scheduled hearing date.
hearings.start_time/end_time	TEXT NOT NULL; CHECK	Hearing interval; end later than start.
tasks.due_date	TEXT NOT NULL	Task deadline.
tasks.status/priority	TEXT defaults	Workflow status and priority.
invoices.invoice_number	TEXT UNIQUE NOT NULL	Unique invoice reference.
invoices.amount	REAL CHECK >= 0	Invoice amount.
payments.amount	REAL CHECK > 0	Payment amount.
expenses.amount	REAL CHECK >= 0	Expense amount.
time_entries.hours	REAL CHECK > 0	Hours recorded against a case.

2.6 IMPLEMENTATION OF LOGICAL DESIGN: DDL AND SAMPLE DATA

The following representative DDL is taken from the project schema. The complete initialization logic is in app.py. It creates tables if they do not already exist and seeds a small set of fictional demonstration records only when selected tables are empty.

```
CREATE TABLE clients (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL, phone TEXT, email TEXT, address TEXT
);

CREATE TABLE cases (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  case_number TEXT NOT NULL UNIQUE, title TEXT NOT NULL,
  client_id INTEGER NOT NULL, case_type_id INTEGER NOT NULL,
  court_id INTEGER, status TEXT NOT NULL DEFAULT 'Open',
  opened_on TEXT NOT NULL, description TEXT,
  FOREIGN KEY(client_id) REFERENCES clients(id),
  FOREIGN KEY(case_type_id) REFERENCES case_types(id),
  FOREIGN KEY(court_id) REFERENCES courts(id)
);

CREATE TABLE assignments (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  case_id INTEGER NOT NULL, lawyer_id INTEGER NOT NULL,
  assigned_on TEXT NOT NULL, UNIQUE(case_id, lawyer_id),
  FOREIGN KEY(case_id) REFERENCES cases(id) ON DELETE CASCADE,
  FOREIGN KEY(lawyer_id) REFERENCES lawyers(id)
);
```

2.6 SQL IMPLEMENTATION AND QUERY EXAMPLES

Invoice and payment constraints

```
CREATE TABLE payments (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  invoice_id INTEGER NOT NULL, paid_on TEXT NOT NULL,
  amount REAL NOT NULL CHECK(amount > 0), method TEXT NOT NULL,
  FOREIGN KEY(invoice_id) REFERENCES invoices(id) ON DELETE CASCADE
);
```

Outstanding balance query

The billing screen calculates the outstanding balance by subtracting recorded payments from the invoice amount. Payments are grouped by invoice before joining, so multiple payment rows are summarized into one paid total.

```
SELECT i.invoice_number, i.amount,
       COALESCE(p.paid, 0) AS paid,
       i.amount - COALESCE(p.paid, 0) AS balance
FROM invoices i
LEFT JOIN (SELECT invoice_id, SUM(amount) AS paid
           FROM payments GROUP BY invoice_id) p
  ON p.invoice_id = i.id;
```

Hearing overlap query

```
SELECT 1 FROM hearings
WHERE lawyer_id = ? AND hearing_date = ?
  AND start_time < ? AND end_time > ?;
```

The interval condition detects overlapping hearings for the same lawyer and date. The application uses parameter placeholders rather than concatenating user input into SQL.

3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

JurisDesk is a local web application implemented with Python, Flask, HTML templates, CSS and SQLite. The Flask application initializes the schema when app.py is run, then exposes routes for the dashboard and record-management modules. Each route uses the database connection helper to run SQL and passes retrieved rows to a Jinja template.

Component	Role
Python 3	Application language and route logic.
Flask	HTTP routing, form processing, template rendering and flash messages.
SQLite	Local relational database stored in legal_management.db.
sqlite3	Python database connector; uses row objects and enables foreign keys.
HTML/Jinja	Forms, navigation, record lists and dashboard components.
CSS	Shared styling in static/style.css.

3.2 Directory structure

```
Legal_Case_Client_Management_App/
  app.py
  requirements.txt
  README.md
  legal_management.db  (created at runtime)
  static/style.css
  templates/
    base.html, dashboard.html, clients.html, lawyers.html
    cases.html, assignments.html, hearings.html, tasks.html
    billing.html, payments.html, reports.html
```

3.2 DEVELOPMENT ENVIRONMENT AND DATABASE CONNECTIVITY

The application uses a connection helper that opens the database file located beside app.py. sqlite3.Row permits access to returned columns by name. Foreign-key enforcement is enabled for each connection.

```
DB = Path(__file__).with_name('legal_management.db')

def db():
    con = sqlite3.connect(DB)
    con.row_factory = sqlite3.Row
    con.execute('PRAGMA foreign_keys=ON')
    return con
```

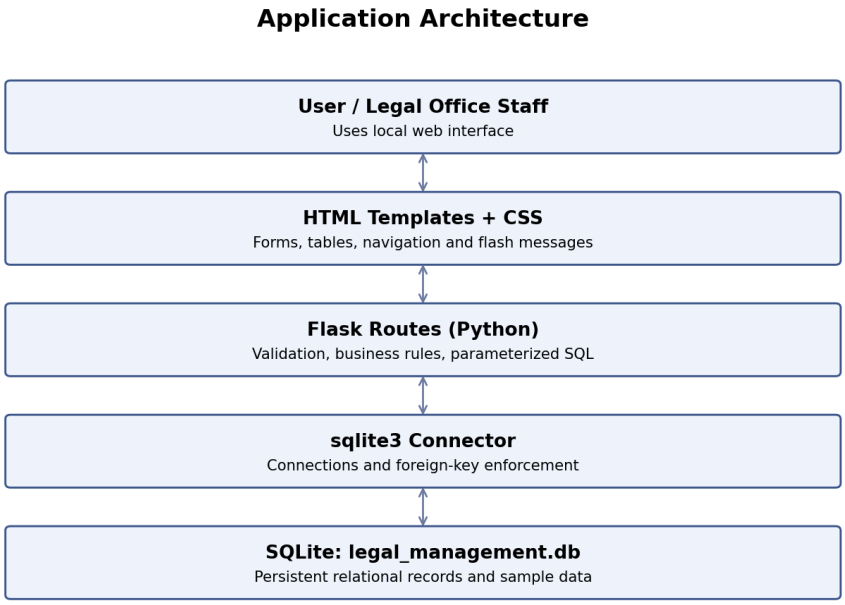


Figure 2. Application architecture from the Flask interface to SQLite storage.

3.3 APPLICATION MODULES AND DATABASE MAPPING

Module	Main tables	Purpose / operations
Dashboard	cases, clients, hearings, invoices, payments	Counts clients and cases, upcoming hearings and outstanding balances.
Clients	clients	Create and list client records.
Lawyers	lawyers, assignments	Create and list lawyers and assigned-case counts.
Cases	cases, clients, case_types, courts	Register and list cases with linked master data.
Assignments	assignments, cases, lawyers	Assign lawyers; duplicate pairs are rejected.
Hearings	hearings, cases, lawyers	Schedule/list hearings and check time overlaps.
Tasks	tasks, cases	Create and list tasks by status and due date.
Billing	invoices, cases, payments	Create invoices and calculate outstanding balances.
Payments	payments, invoices, cases	Record payments and update invoice status.
Reports	lawyers, assignments, time_entries, cases, hearings, invoices, payments	Summaries of lawyer activity, case status, outstanding invoices and upcoming hearings.

3.4 CRUD Operations

Create and Read operations are available in the main modules listed above. Update and Delete operations are not uniformly provided through the visible UI, so the report does not claim full CRUD coverage for every entity. Some payment processing updates the invoice status after a successful payment.

3.5 BUSINESS RULES, VALIDATION AND ERROR HANDLING

Rule	Implementation	Expected behaviour
Unique case number	UNIQUE constraint and caught IntegrityError	Duplicate case registration is rejected.
Unique lawyer-case assignment	UNIQUE(case_id, lawyer_id)	Same lawyer cannot be added to same case twice.
Hearing interval	CHECK(end_time > start_time) and route validation	End time must follow start time.
Hearing conflict	Query checks same lawyer/date and overlapping intervals	Overlapping hearing is blocked.
Invoice number	UNIQUE constraint	Duplicate invoice number is rejected.
Payment amount	Must be positive and no more than balance	Invalid overpayment is blocked.
Invoice status	Application sets Paid or Partially Paid after payment	Status reflects the remaining amount.
Foreign keys	PRAGMA foreign_keys=ON and schema declarations	References must point to existing parent rows.

The prototype uses flash messages to show success or validation errors. Error handling is implemented for selected integrity constraints, while broader production-grade exception logging and input validation remain future improvements.

3.6 REPORTS AND DATABASE OPERATIONS

Report	SQL concepts	Purpose
Dashboard totals	COUNT, SUM, COALESCE, scalar subqueries	Show current case/client counts and unpaid amount.
Lawyer activity	LEFT JOIN, COUNT DISTINCT, SUM, GROUP BY	Summarize assigned cases and recorded hours.
Case status	GROUP BY status	Count cases by workflow status.
Outstanding invoices	LEFT JOIN aggregated payments	List invoices with positive remaining balances.
Upcoming hearings	JOIN, WHERE date >= today, ORDER BY	Provide an ordered hearing list.
Billing balance	SUM payments per invoice	Calculate amount paid and amount remaining.

3.7 Application Screens

The supplied application contains templates for the dashboard, clients, lawyers, cases, assignments, hearings, tasks, billing, payments and reports. For submission, capture screenshots from the running local application rather than substituting illustrative mockups. Suggested evidence includes the dashboard, a case register, the hearing form with conflict feedback, and the invoice/payment view.

3.8 TESTING AND RESULTS

Evidence from the source code confirms that validation rules are implemented. The table below separates code-supported behaviour from runtime tests that should be executed and captured before final submission.

ID	Test	Expected result	Status
TC-01	Start app and open dashboard	Dashboard loads from SQLite.	Run on final folder
TC-02	Register a new client	Client appears in the client list.	To verify
TC-03	Register duplicate case number	Case is rejected with validation message.	To verify
TC-04	Assign same lawyer to same case twice	Second assignment is rejected.	To verify
TC-05	Create overlapping hearings for one lawyer	Second hearing is blocked.	To verify
TC-06	Create invoice with duplicate number	Invoice is rejected.	To verify
TC-07	Record payment larger than invoice balance	Payment is rejected.	To verify
TC-08	Record valid partial payment	Payment is recorded; invoice status becomes Partially Paid.	To verify
TC-09	Pay remaining balance exactly	Invoice status becomes Paid.	To verify
TC-10	Open reports page	Status, outstanding balance and upcoming hearing reports render.	To verify

3.8 TESTING AND RESULTS — DISCUSSION

Observed design outcomes

- The schema represents master data and case-related transactions in separate tables.
- Foreign keys link cases to clients, case types and courts, and link hearings and tasks to cases.
- Unique constraints protect case numbers, invoice numbers, case types and lawyer-case assignments.
- The hearing route checks interval overlap for the same lawyer and hearing date.
- The payment route compares an entered payment with the outstanding invoice balance and updates invoice status.
- The report route combines joins and aggregate queries to present case and billing summaries.

Verification still required

The source code establishes the intended behaviour but does not by itself prove that every test has been executed successfully on the target computer. The final demonstration should run each test with sample data and record the actual message or database change. Only then should the status be changed to Passed.

4. CONCLUSION

JurisDesk demonstrates the use of a relational database to organize legal case and client information. The schema separates clients, lawyers, case types, courts, cases, assignments, hearings, tasks, documents, time entries, expenses, invoices and payments. Primary keys, foreign keys, unique constraints and check constraints provide a foundation for data integrity.

The Python Flask interface connects to SQLite and provides record-entry and listing workflows for the main modules. Application-level validation prevents selected problems such as duplicate assignments, overlapping hearings for the same lawyer and payments greater than the outstanding invoice balance. Summary queries support monitoring of case status, upcoming hearings and unpaid invoices.

4.1 Limitations and Future Enhancements

- Implement authentication, role-based access and session security.
- Add edit/delete workflows and an audit log for important changes.
- Provide secure document upload, access controls and backup/restore.
- Add reminders for hearings, tasks and invoice due dates.
- Improve date/time validation and centralize error handling.
- Test concurrent access and prepare deployment configuration before any real use.

The prototype is suitable for demonstrating DBMS concepts in a controlled academic environment. It must not be used to store real confidential legal records without substantial security, privacy, backup and operational improvements.

5. REFERENCES

[1] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” Python Documentation. [Online]. Available: https://docs.python.org/3/library/sqlite3.html. Accessed: Oct. 12, 2026.

[2] Pallets Projects, “Flask Documentation.” [Online]. Available: https://flask.palletsprojects.com/. Accessed: Oct. 12, 2026.

[3] SQLite Consortium, “SQLite Documentation.” [Online]. Available: https://www.sqlite.org/docs.html. Accessed: Oct. 12, 2026.

[4] Python Software Foundation, “sqlite3 — SQLite database API,” Python Documentation. [Online]. Available: https://docs.python.org/3/library/sqlite3.html. Accessed: Oct. 12, 2026.

[5] Woxsen University, School of Technology, “DBMS Project-Based Learning Report Template and Report Preparation Guidelines,” course-provided template, 2026.

[6] JurisDesk project source code, app.py and HTML templates, supplied academic prototype, Oct. 2026.

6. APPENDICES

Appendix A. Representative Database Schema

The following extract demonstrates the schema style. The complete database initialization script is in app.py; use that file as the authoritative source when submitting code.

```
CREATE TABLE lawyers (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 name TEXT NOT NULL, specialization TEXT NOT NULL,
 email TEXT, hourly_rate REAL NOT NULL DEFAULT 1000
   CHECK(hourly_rate >= 0)
);

CREATE TABLE invoices (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 invoice_number TEXT NOT NULL UNIQUE, case_id INTEGER NOT NULL,
 issued_on TEXT NOT NULL, due_date TEXT NOT NULL,
 amount REAL NOT NULL CHECK(amount >= 0),
 status TEXT NOT NULL DEFAULT 'Unpaid',
 FOREIGN KEY(case_id) REFERENCES cases(id)
);
```

Appendix B. Example seed records

The source initializes illustrative case types (Civil, Criminal, Corporate, Family and Property), sample courts in Hyderabad, fictional clients and lawyers, sample cases, hearings, tasks, time entries, expenses, invoices and a payment. The records exist to make the local demonstration immediately usable.

APPENDIX C. SQL QUERY CATALOGUE AND USER GUIDE

C.1 Upcoming hearings

```
SELECT h.hearing_date, h.start_time, c.case_number,
       c.title, l.name AS lawyer
FROM hearings h
JOIN cases c ON c.id = h.case_id
JOIN lawyers l ON l.id = h.lawyer_id
WHERE h.hearing_date >= ?
ORDER BY h.hearing_date, h.start_time;
```

C.2 Case status summary

```
SELECT status, COUNT(*) AS case_count
FROM cases GROUP BY status;
```

C.3 Run the application

- Open the Legal_Case_Client_Management_App folder in VS Code.
- Install dependencies with: python -m pip install -r requirements.txt
- Run: python app.py
- Open the local URL printed by Flask, usually http://127.0.0.1:5000.
- Keep the terminal open during the demonstration; stop with Ctrl+C.

The database file legal_management.db is created automatically beside app.py when the app starts. Open that exact file in DB Browser for SQLite to inspect tables and records.

APPENDIX D. FINAL DEMONSTRATION CHECKLIST

Step	Demonstration action	Evidence to capture
1	Open dashboard	Dashboard counters and upcoming hearings.
2	Open clients and lawyers	Sample records and add forms.
3	Register or inspect a case	Case number, client, type, court and status.
4	Assign lawyer to case	Assignment appears in list.
5	Attempt overlapping hearing	Validation message; no duplicate hearing inserted.
6	Open tasks	Task priority, status and due date.
7	Open billing and payments	Invoice amount, paid amount and remaining balance.
8	Open reports	Case status, outstanding invoices and upcoming hearings.
9	Open DB Browser	Show legal_management.db and the relevant tables.
10	Check submitted files	app.py, requirements.txt, templates/, static/ and database file if required.

Submission checklist

- Replace any sample screenshot placeholders with genuine screenshots from the running app.
- Verify the actual test outcomes and update the status column.
- Ensure the report's ER diagram and schema remain consistent with app.py.
- Include the source code folder or ZIP with the report.
- Do not enter real confidential client information into the demonstration database.